Atelier 3

Gestion d'un Centre de Formation avec Procédures et Fonctions Stockées

Rapport technique

Réalisé par	Yassmina Aouladhaj
Sujet	Base de données MySQL « TechLearn » : procédures, fonctions, paramètres IN / OUT / INOUT, boucles et conditions
Environnement	MySQL 8.4.11 hébergé sur Aiven, client en ligne de commande (CMD Windows)
Date	12 novembre 2026

Sommaire

Sommaire 2

1. Contexte et objectifs pédagogiques 3

Objectifs pédagogiques . . . 3

2. Environnement de travail 3

3. Création des tables 4

4. Insertion des données 5

5. Exercices 7

Exercice 1 – Procédure avec paramètre IN . . . 7

Exercice 2 – Procédure avec paramètre OUT . . . 7

Exercice 3 – Procédure avec paramètre INOUT . . . 8

Exercice 4 – Fonction : taux d'occupation d'une session . . . 9

Exercice 5 – Boucle et automatisation . . . 10

6. Défis finaux 13

Défi 1 – Fonction sessions_formateur . . . 13

Défi 2 – Procédure supprimer_sessions_vides . . . 13

Défi 3 – Procédure valider_inscription . . . 13

7. Vérification des objets créés 15

8. Analyse globale 16

Les trois modes de paramètres . . . 16

Procédure ou fonction ? . . . 16

Structures de contrôle utilisées . . . 16

9. Conclusion personnelle 16

1. Contexte et objectifs pédagogiques

Le centre de formation **TechLearn** propose plusieurs sessions animées par des formateurs spécialisés. Chaque session possède un formateur, une capacité maximale et un nombre d'inscrits. L'administration souhaite automatiser plusieurs tâches : le calcul du taux d'occupation des sessions, la mise à jour du nombre d'inscrits, la suppression des sessions vides et la génération de rapports.

Cet atelier consiste à concevoir et manipuler une base de données MySQL qui répond à ces besoins, en utilisant des **procédures stockées** et des **fonctions stockées**.

Objectifs pédagogiques

- Créer une base de données relationnelle (formateurs, stagiaires, sessions) avec clés primaires et clé étrangère.
- Écrire des procédures stockées avec les paramètres **IN**, **OUT** et **INOUT**.
- Écrire des fonctions stockées qui retournent une valeur utilisable dans une requête SELECT.
- Utiliser les structures de contrôle : boucle `WHILE` et condition `IF / ELSE`.
- Documenter et présenter les résultats dans un rapport technique.

2. Environnement de travail

Le travail a été réalisé avec le client `mysql` en ligne de commande sous Windows, connecté à un service MySQL hébergé dans le cloud chez **Aiven** (connexion sécurisée SSL, option `--ssl-mode=REQUIRED`).

Élément	Valeur
Serveur	MySQL 8.4.11 (Aiven, service « atelie1module3 »)
Client	mysql en ligne de commande (CMD Windows)
Utilisateur	avnadmin
Base utilisée	`defaultdb` (base fournie par défaut par Aiven)

Remarque : le sujet prévoit `CREATE DATABASE techlearn_db`. Sur Aiven, une base `defaultdb` est déjà fournie ; nous l'avons utilisée directement et les étapes `CREATE DATABASE` / `USE` ont donc été omises. Le reste du code est identique à celui du sujet.

Commande de connexion utilisée :

```
mysql -h atelie1module3-aouladhajyassmina-60c1.h.aivencloud.com -P 28703 -u avnadmin -p --ssl-mode=REQUIRED defaultd
```

Un point important : dans le client MySQL, le caractère `;` termine une instruction. Comme le corps d'une procédure contient lui-même des `;`, on change temporairement le délimiteur avec `DELIMITER //` avant la création, puis on le rétablit avec `DELIMITER ;`.

3. Création des tables

Trois tables sont créées : **formateurs**, **stagiaires** et **sessions**. La table *sessions* est reliée à *formateurs* par la clé étrangère `formateur_id`.

```
CREATE TABLE formateurs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  specialite VARCHAR(100)
);

CREATE TABLE stagiaires (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  ville VARCHAR(100)
);

CREATE TABLE sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  intitule VARCHAR(100),
  formateur_id INT,
  capacite INT,
  inscrits INT DEFAULT 0,
  FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
);
```

Avant la création, `SHOW TABLES` retourne « Empty set » : la base est vide. Après les trois `CREATE TABLE` (chacun répond *Query OK*), `SHOW TABLES` affiche bien les trois tables.

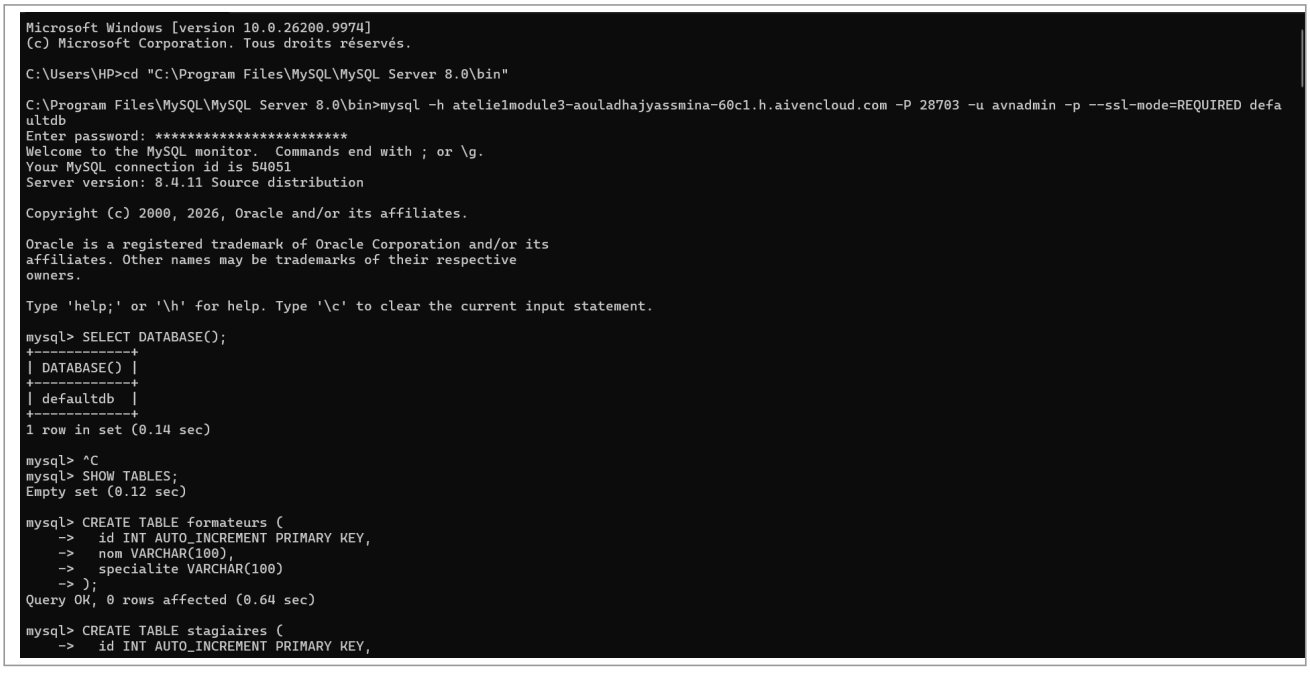


Figure 1 – Connexion à Aiven, vérification de la base (defaultdb) et début de la création des tables.

```
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(100),
    ->   ville VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.53 sec)

mysql> CREATE TABLE sessions (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   intitule VARCHAR(100),
    ->   formateur_id INT,
    ->   capacite INT,
    ->   inscrits INT DEFAULT 0,
    ->   FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
    -> );
Query OK, 0 rows affected (0.16 sec)

mysql> SHOW TABLES;
+--------------------+
| Tables_in_defaultdb |
+--------------------+
| formateurs          |
| sessions            |
| stagiaires          |
+--------------------+
3 rows in set (0.11 sec)

mysql> INSERT INTO formateurs (nom, specialite) VALUES
    -> ('Mme Nadia', 'Bases de données'),
    -> ('M. Karim', 'Développement Web'),
    -> ('Mme Salma', 'Analyse de données');
Query OK, 3 rows affected (0.16 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO stagiaires (nom, ville) VALUES
    -> ('Ali', 'Casablanca'),
    -> ('Sara', 'Rabat'),
    -> ('Mouna', 'Marrakech');
Query OK, 3 rows affected (0.11 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
```

Figure 2 – Fin de la création des tables, SHOW TABLES (3 tables) et insertion des données.

4. Insertion des données

Les données de test du sujet sont insérées : 3 formateurs, 3 stagiaires et 3 sessions.

```
INSERT INTO formateurs (nom, specialite) VALUES
('Mme Nadia', 'Bases de données'),
('M. Karim', 'Développement Web'),
('Mme Salma', 'Analyse de données');

INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Mouna', 'Marrakech');

INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
('MySQL avancé', 1, 20, 5),
('HTML/CSS', 2, 25, 10),
('Python Data', 3, 15, 8);
```

Chaque insertion retourne « 3 rows affected » sans avertissement. Les trois `SELECT * FROM ...` confirment le contenu des tables (Figure 3).

```
    -> ('MySQL avancé', 1, 20, 5),
    -> ('HTML/CSS', 2, 25, 10),
    -> ('Python Data', 3, 15, 8);
Query OK, 3 rows affected (0.14 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM formateurs;
+----+----------+--------------------+
| id | nom      | specialite         |
+----+----------+--------------------+
|  1 | Mme Nadia | Bases de données    |
|  2 | M. Karim  | Développement Web   |
|  3 | Mme Salma | Analyse de données  |
+----+----------+--------------------+
3 rows in set (0.17 sec)

mysql> SELECT * FROM stagiaires;
+----+-------+------------+
| id | nom   | ville      |
+----+-------+------------+
|  1 | Ali   | Casablanca |
|  2 | Sara  | Rabat      |
|  3 | Mouna | Marrakech  |
+----+-------+------------+
3 rows in set (0.05 sec)

mysql> SELECT * FROM sessions;
+----+--------------+--------------+----------+----------+
| id | intitule     | formateur_id | capacite | inscrits |
+----+--------------+--------------+----------+----------+
|  1 | MySQL avancé |            1 |       20 |        5 |
|  2 | HTML/CSS     |            2 |       25 |       10 |
|  3 | Python Data  |            3 |       15 |        8 |
+----+--------------+--------------+----------+----------+
3 rows in set (0.11 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
    -> BEGIN
    ->   SELECT * FROM stagiaires WHERE ville = ville_nom;
    -> END //
```

*Figure 3 – Insertion des sessions et contenu des trois tables (SELECT *).*

5. Exercices

Exercice 1 – Procédure avec paramètre IN

Objectif : afficher la liste des stagiaires d'une ville donnée.

```
DELIMITER //
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
BEGIN
  SELECT * FROM stagiaires WHERE ville = ville_nom;
END //
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
CALL afficher_stagiaires_par_ville('Rabat');
```

Résultat : l'appel avec « Casablanca » retourne *Ali* et l'appel avec « Rabat » retourne *Sara*.

Analyse : le paramètre `IN` permet de transmettre une valeur à la procédure. Ici, la ville fournie est utilisée dans la clause `WHERE` pour filtrer les stagiaires : le même code sert pour n'importe quelle ville.

Exercice 2 – Procédure avec paramètre OUT

Objectif : retourner le nombre total de sessions.

```
DELIMITER //
CREATE PROCEDURE compter_sessions(OUT total INT)
BEGIN
  SELECT COUNT(*) INTO total FROM sessions;
END //
DELIMITER ;

CALL compter_sessions(@nb);
SELECT @nb AS nombre_total_sessions;
```

Résultat : `nombre_total_sessions = 3`.

Analyse : avec `OUT`, c'est la procédure qui renvoie une valeur à l'appelant. On passe une variable de session (`@nb`) que la procédure remplit grâce à `SELECT ... INTO` ; on la lit ensuite avec un `SELECT`.

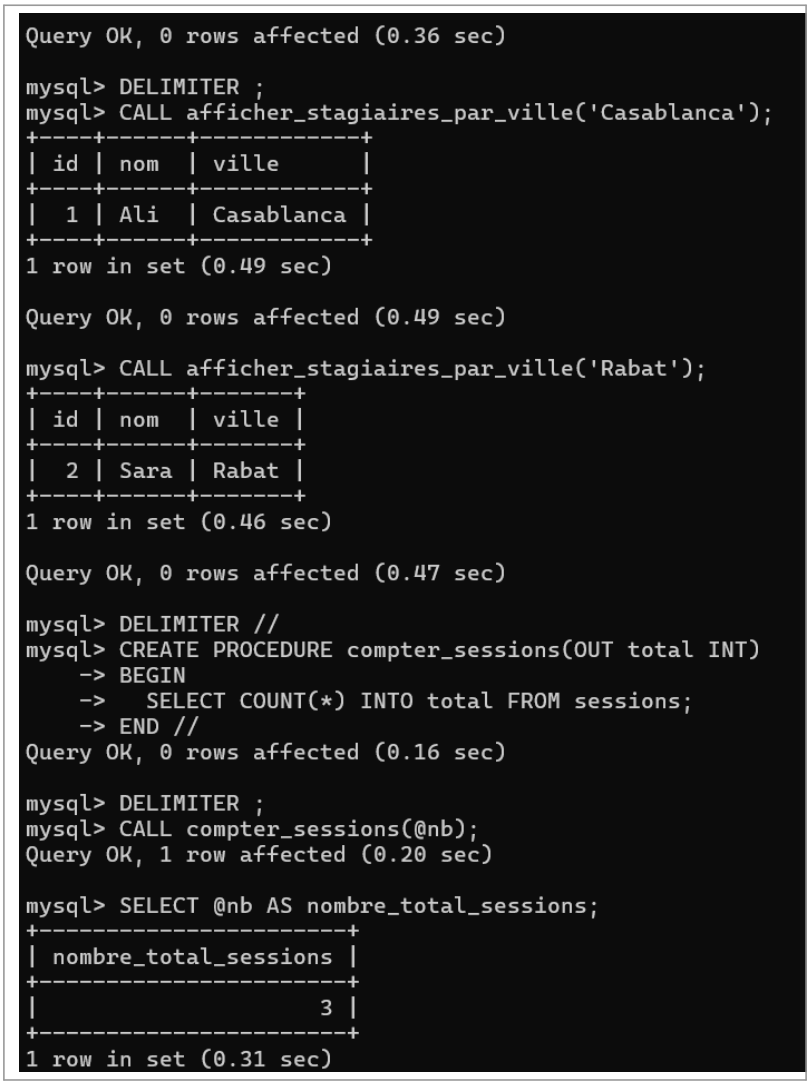


Figure 4 – Exercices 1 et 2 : création, appels et résultats.

Exercice 3 – Procédure avec paramètre INOUT

Objectif : incrémenter le nombre d'inscrits d'une session et retourner le nouveau total.

```
DELIMITER //
CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
BEGIN
  UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
  SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
END //
DELIMITER ;

SET @nouveau = 0;
CALL inscrire_stagiaire(@nouveau, 1);
SELECT @nouveau AS nouveau_nombre_inscrits;
```

Résultat : `nouveau_nombre_inscrits = 6`. La session 1 « MySQL avancé » passe de 5 à 6 inscrits, ce que confirme le `SELECT * FROM sessions`.

Analyse : `INOUT` combine les deux sens : la variable est transmise à la procédure (ici 0) puis écrasée par la valeur calculée (6) et renvoyée à l'appelant. La procédure modifie aussi la table (`UPDATE`).

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
    -> BEGIN
    ->   UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    ->   SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
    -> END //
Query OK, 0 rows affected (0.11 sec)

mysql> DELIMITER ;
mysql> SET @nouveau = 0;
Query OK, 0 rows affected (0.20 sec)

mysql> CALL inscrire_stagiaire(@nouveau, 1);
Query OK, 1 row affected (0.10 sec)

mysql> SELECT @nouveau AS nouveau_nombre_inscrits;
+-------------------------+
| nouveau_nombre_inscrits |
+-------------------------+
|                       6 |
+-------------------------+
1 row in set (0.15 sec)

mysql> SELECT * FROM sessions;
+----+---------------+--------------+----------+----------+
| id | intitule      | formateur_id | capacite | inscrits |
+----+---------------+--------------+----------+----------+
|  1 | MySQL avancé  |            1 |       20 |        6 |
|  2 | HTML/CSS      |            2 |       25 |       10 |
|  3 | Python Data   |            3 |       15 |        8 |
+----+---------------+--------------+----------+----------+
3 rows in set (0.48 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION taux_occupation(id_session INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE taux DECIMAL(5,2);
```

Figure 5 – Exercice 3 : INOUT, résultat 6 et table sessions mise à jour.

Exercice 4 – Fonction : taux d'occupation d'une session

Objectif : calculer le pourcentage d'occupation d'une session (inscrits / capacité × 100).

```
DELIMITER //
CREATE FUNCTION taux_occupation(id_session INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE taux DECIMAL(5,2);
  SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE id = id_session;
  RETURN taux;
END //
DELIMITER ;

SELECT intitule, taux_occupation(id) AS taux FROM sessions;
```

Session	Calcul	Taux
MySQL avancé	6 / 20	30.00 %
HTML/CSS	10 / 25	40.00 %
Python Data	8 / 15	53.33 %

Analyse : une fonction retourne une **seule valeur** (`RETURNS DECIMAL(5,2)`) et s'utilise directement dans un `SELECT`, ce qui n'est pas possible avec une procédure (qui s'appelle avec `CALL`). Les caractéristiques `DETERMINISTIC READS SQL DATA` sont déclarées car le serveur MySQL exige de préciser la nature de la fonction lorsque la journalisation binaire est activée.

Exercice 5 – Boucle et automatisation

Objectif : créer automatiquement 10 sessions pour un formateur donné.

```
DELIMITER //
CREATE PROCEDURE generer_sessions(IN formateur INT)
BEGIN
  DECLARE i INT DEFAULT 1;
  WHILE i <= 10 DO
    INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;

CALL generer_sessions(2);
SELECT * FROM sessions;
```

Résultat : la table contient désormais 13 sessions : les 3 sessions d'origine et 10 nouvelles (id 4 à 13), toutes rattachées au formateur 2, avec une capacité de 20 et 0 inscrit.

Analyse : la boucle `WHILE` répète l'`INSERT` tant que `i <= 10`. `CONCAT` construit l'intitulé de chaque session et `SET i = i + 1` incrémente le compteur ; sans cette ligne, la boucle serait infinie.

```
    ->   SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE id = id_session;
    ->   RETURN taux;
    -> END //
Query OK, 0 rows affected (0.18 sec)

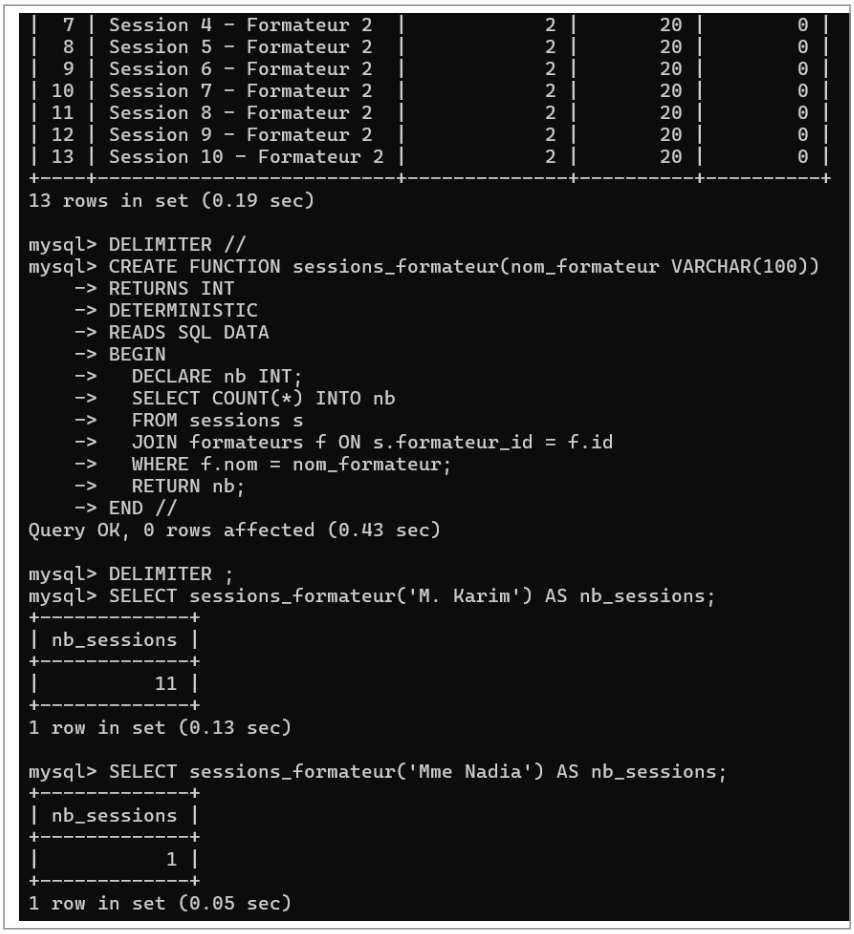
mysql> DELIMITER ;
mysql> SELECT intitule, taux_occupation(id) AS taux FROM sessions;
+---------------+-------+
| intitule      | taux  |
+---------------+-------+
| MySQL avancé  | 30.00 |
| HTML/CSS      | 40.00 |
| Python Data   | 53.33 |
+---------------+-------+
3 rows in set (0.13 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE generer_sessions(IN formateur INT)
    -> BEGIN
    ->   DECLARE i INT DEFAULT 1;
    ->   WHILE i <= 10 DO
    ->     INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    ->     VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    ->     SET i = i + 1;
    ->   END WHILE;
    -> END //
Query OK, 0 rows affected (0.29 sec)

mysql> DELIMITER ;
mysql> CALL generer_sessions(2);
Query OK, 1 row affected (0.12 sec)

mysql> SELECT * FROM sessions;
+----+--------------------------+--------------+----------+----------+
| id | intitule                 | formateur_id | capacite | inscrits |
+----+--------------------------+--------------+----------+----------+
|  1 | MySQL avancé             |            1 |       20 |        6 |
|  2 | HTML/CSS                 |            2 |       25 |       10 |
|  3 | Python Data              |            3 |       15 |        8 |
|  4 | Session 1 - Formateur 2  |            2 |       20 |        0 |
|  5 | Session 2 - Formateur 2  |            2 |       20 |        0 |
```

Figure 6 – Exercices 4 et 5 : fonction taux_occupation et procédure generer_sessions.



```
|  7 | Session 4 - Formateur 2  |            2 |       20 |        0 |
|  8 | Session 5 - Formateur 2  |            2 |       20 |        0 |
|  9 | Session 6 - Formateur 2  |            2 |       20 |        0 |
| 10 | Session 7 - Formateur 2  |            2 |       20 |        0 |
| 11 | Session 8 - Formateur 2  |            2 |       20 |        0 |
| 12 | Session 9 - Formateur 2  |            2 |       20 |        0 |
| 13 | Session 10 - Formateur 2 |            2 |       20 |        0 |
+----+--------------------------+--------------+----------+----------+
13 rows in set (0.19 sec)

mysql> DELIMITER //
mysql> CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
    -> RETURNS INT
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->   DECLARE nb INT;
    ->   SELECT COUNT(*) INTO nb
    ->   FROM sessions s
    ->   JOIN formateurs f ON s.formateur_id = f.id
    ->   WHERE f.nom = nom_formateur;
    ->   RETURN nb;
    -> END //
Query OK, 0 rows affected (0.43 sec)

mysql> DELIMITER ;
mysql> SELECT sessions_formateur('M. Karim') AS nb_sessions;
+-------------+
| nb_sessions |
+-------------+
|          11 |
+-------------+
1 row in set (0.13 sec)

mysql> SELECT sessions_formateur('Mme Nadia') AS nb_sessions;
+-------------+
| nb_sessions |
+-------------+
|           1 |
+-------------+
1 row in set (0.05 sec)
```

Figure 7 – Fin de l'exercice 5 (13 sessions) et défi 1 : fonction sessions_formateur.

6. Défis finaux

Défi 1 – Fonction sessions_formateur

Objectif : retourner le nombre de sessions animées par un formateur donné (recherché par son nom).

```
DELIMITER //
CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
RETURNS INT
DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE nb INT;
  SELECT COUNT(*) INTO nb
  FROM sessions s
  JOIN formateurs f ON s.formateur_id = f.id
  WHERE f.nom = nom_formateur;
  RETURN nb;
END //
DELIMITER ;

SELECT sessions_formateur('M. Karim') AS nb_sessions;
SELECT sessions_formateur('Mme Nadia') AS nb_sessions;
```

Résultat : 11 sessions pour M. Karim (1 d'origine + 10 générées) et 1 session pour Mme Nadia.

Analyse : la fonction fait une jointure entre *sessions* et *formateurs* pour retrouver le formateur à partir de son nom, puis compte ses sessions avec `COUNT(*)`.

Défi 2 – Procédure supprimer_sessions_vides

Objectif : supprimer les sessions qui n'ont aucun inscrit.

```
DELIMITER //
CREATE PROCEDURE supprimer_sessions_vides()
BEGIN
  DELETE FROM sessions WHERE inscrits = 0;
END //
DELIMITER ;

CALL supprimer_sessions_vides();
SELECT * FROM sessions;
```

Résultat : « 10 rows affected » : les 10 sessions générées à l'exercice 5 (0 inscrit) sont supprimées ; il reste les 3 sessions d'origine.

Analyse : cette procédure sans paramètre automatise un nettoyage de la base. Elle montre l'intérêt d'enchaîner les exercices : la procédure de l'exercice 5 crée des sessions vides, celle-ci les élimine.

Défi 3 – Procédure valider_inscription

Objectif : si la session est complète, afficher « Session complète » ; sinon ajouter un inscrit et afficher « Inscription validée ».

```
DELIMITER //
CREATE PROCEDURE valider_inscription(IN session_id INT)
BEGIN
  DECLARE v_inscrits INT;
  DECLARE v_capacite INT;

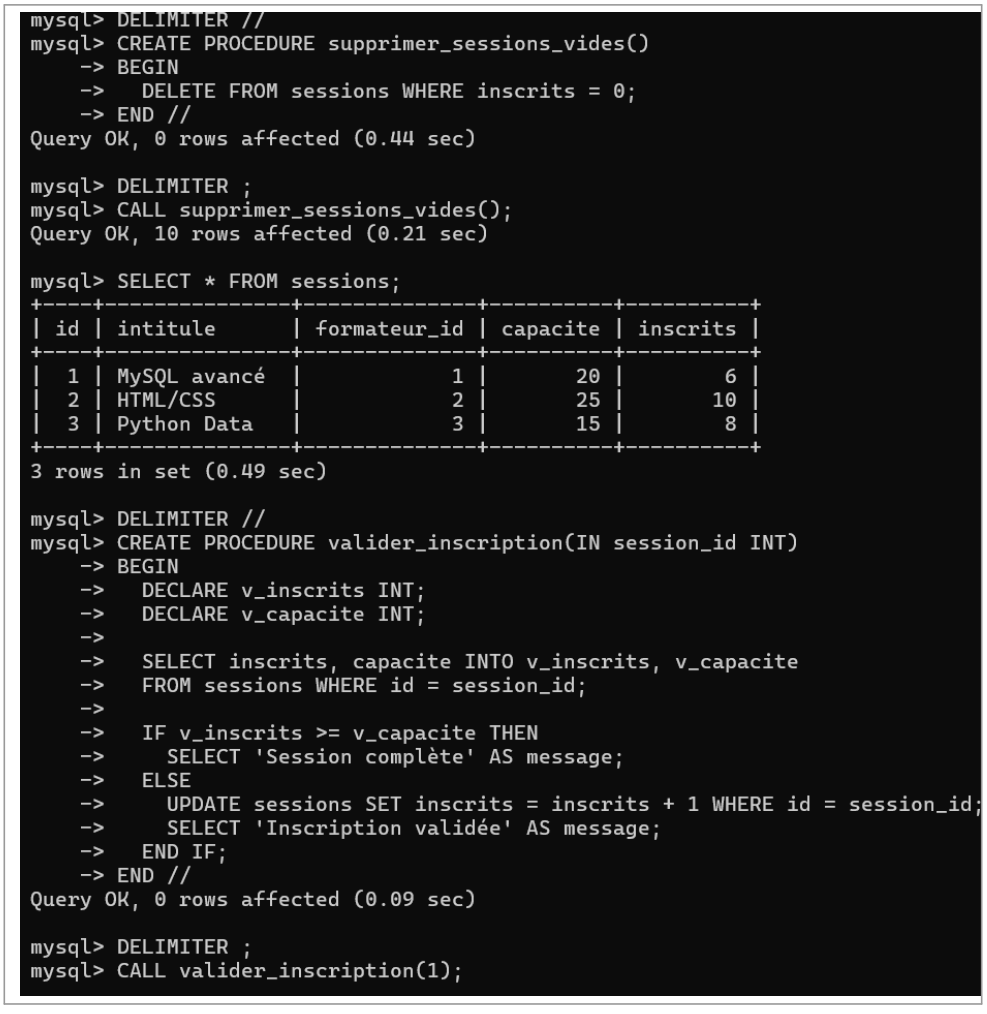
  SELECT inscrits, capacite INTO v_inscrits, v_capacite
  FROM sessions WHERE id = session_id;
```

```
  IF v_inscrits >= v_capacite THEN
    SELECT 'Session complète' AS message;
  ELSE
    UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    SELECT 'Inscription validée' AS message;
  END IF;
END //
DELIMITER ;

CALL valider_inscription(1);
UPDATE sessions SET inscrits = 15 WHERE id = 3;
CALL valider_inscription(3);
```

Test	Situation	Résultat affiché
CALL valider_inscription(1)	6 / 20 → place disponible	Inscription validée (inscrits passe à 7)
CALL valider_inscription(3)	15 / 15 après UPDATE → complète	Session complète

Analyse : la procédure lit d'abord les valeurs dans des variables locales (`DECLARE` + `SELECT ... INTO`), puis une structure `IF / ELSE` choisit le comportement. Le test de la session 3 a nécessité de la remplir au préalable avec un `UPDATE` pour atteindre le cas « complète ».



```
mysql> DELIMITER //
mysql> CREATE PROCEDURE supprimer_sessions_vides()
    -> BEGIN
    ->   DELETE FROM sessions WHERE inscrits = 0;
    -> END //
Query OK, 0 rows affected (0.44 sec)

mysql> DELIMITER ;
mysql> CALL supprimer_sessions_vides();
Query OK, 10 rows affected (0.21 sec)

mysql> SELECT * FROM sessions;
+----+---------------+--------------+----------+----------+
| id | intitule      | formateur_id | capacite | inscrits |
+----+---------------+--------------+----------+----------+
|  1 | MySQL avancé  |            1 |       20 |        6 |
|  2 | HTML/CSS      |            2 |       25 |       10 |
|  3 | Python Data   |            3 |       15 |        8 |
+----+---------------+--------------+----------+----------+
3 rows in set (0.49 sec)

mysql> DELIMITER //
mysql> CREATE PROCEDURE valider_inscription(IN session_id INT)
    -> BEGIN
    ->   DECLARE v_inscrits INT;
    ->   DECLARE v_capacite INT;
    ->
    ->   SELECT inscrits, capacite INTO v_inscrits, v_capacite
    ->   FROM sessions WHERE id = session_id;
    ->
    ->   IF v_inscrits >= v_capacite THEN
    ->     SELECT 'Session complète' AS message;
    ->   ELSE
    ->     UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    ->     SELECT 'Inscription validée' AS message;
    ->   END IF;
    -> END //
Query OK, 0 rows affected (0.09 sec)

mysql> DELIMITER ;
mysql> CALL valider_inscription(1);
```

Figure 8 – Défis 2 et 3 : suppression des sessions vides et création de valider_inscription.

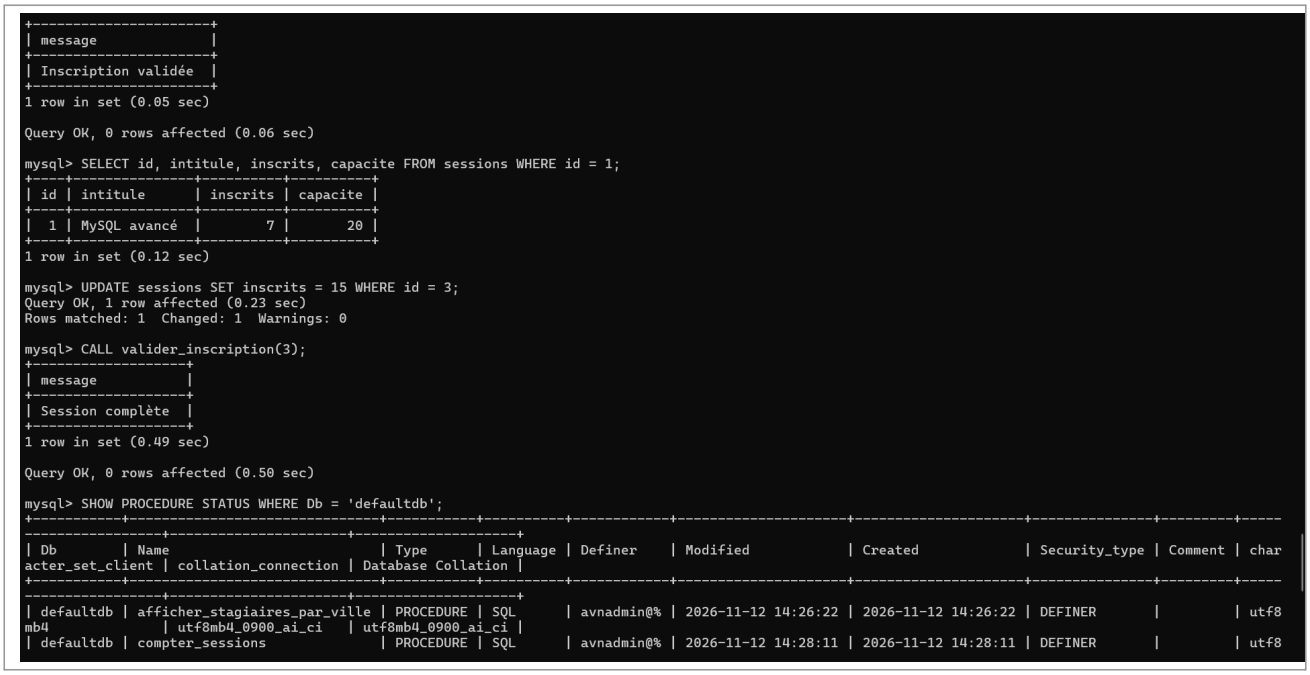


Figure 9 – Défi 3 : résultats « Inscription validée » et « Session complète ».

7. Vérification des objets créés

Les commandes suivantes listent les procédures et fonctions présentes dans la base :

```
SHOW PROCEDURE STATUS WHERE Db = 'defaultdb';
SHOW FUNCTION STATUS WHERE Db = 'defaultdb';
```

Type	Nom	Rôle
Procédure	afficher_stagiaires_par_ville	Filtre les stagiaires par ville (IN)
Procédure	compter_sessions	Retourne le nombre de sessions (OUT)
Procédure	inscrire_stagiaire	Ajoute un inscrit et retourne le total (INOUT)
Procédure	generer_sessions	Crée 10 sessions avec une boucle WHILE
Procédure	supprimer_sessions_vides	Supprime les sessions sans inscrit
Procédure	valider_inscription	Valide une inscription selon la capacité (IF/ELSE)
Fonction	taux_occupation	Calcule le taux d'occupation d'une session
Fonction	sessions_formateur	Compte les sessions d'un formateur

Le résultat confirme la présence des **6 procédures** et des **2 fonctions**.

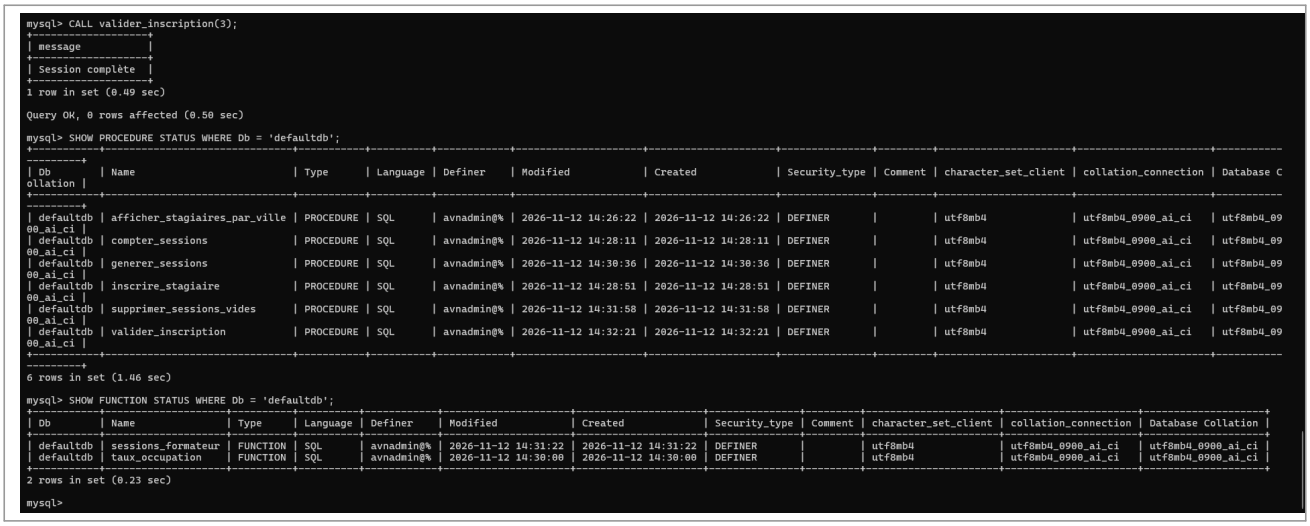


Figure 10 – SHOW PROCEDURE STATUS (6 procédures) et SHOW FUNCTION STATUS (2 fonctions).

8. Analyse globale

Les trois modes de paramètres

Mode	Sens de la valeur	Exemple dans l'atelier
IN	De l'appelant vers la procédure (lecture seule)	ville_nom, session_id
OUT	De la procédure vers l'appelant	total dans compter_sessions
INOUT	Dans les deux sens	nb_inscrits dans inscrire_stagiaire

Procédure ou fonction ?

	Procédure	Fonction
Appel	`CALL nom(...)`	Dans une expression : `SELECT nom(...)`
Valeur retournée	Aucune obligatoire ; peut utiliser OUT / INOUT	Une seule valeur avec RETURN
Usage typique	Actions : INSERT, UPDATE, DELETE, traitements	Calculs réutilisables dans des requêtes
Exemples	generer_sessions, valider_inscription	taux_occupation, sessions_formateur

Structures de contrôle utilisées

- **WHILE** (exercice 5) : répéter une action un nombre défini de fois.
- **IF / ELSE** (défi 3) : exécuter un traitement différent selon une condition.
- **Variables locales** (`DECLARE`) : stocker des valeurs intermédiaires dans une procédure ou une fonction.

9. Conclusion personnelle

Cet atelier m'a permis de comprendre concrètement comment déplacer une partie de la logique métier directement dans la base de données. Au lieu de réécrire les mêmes requêtes à chaque fois, une procédure ou une fonction stockée les encapsule sous un nom unique, ce qui rend les traitements plus simples à appeler, à réutiliser et à maintenir.

J'ai surtout retenu la différence entre les paramètres **IN**, **OUT** et **INOUT**, ainsi que celle entre une procédure (qui agit) et une fonction (qui calcule et retourne une valeur utilisable dans un SELECT).

La boucle `WHILE` et la condition `IF / ELSE` m'ont montré comment automatiser des tâches comme la génération de sessions ou la validation d'une inscription.

Sur le plan pratique, j'ai rencontré quelques difficultés : l'usage de `DELIMITER` dans le terminal, la précision des caractéristiques `DETERMINISTIC READS SQL DATA` pour les fonctions, et la prise en main d'un serveur MySQL hébergé sur Aiven via la ligne de commande. Les surmonter m'a appris à lire les messages d'erreur et à vérifier chaque étape avec des requêtes de contrôle. J'ai aussi pris conscience de l'importance de l'ordre d'exécution : les résultats des défis dépendent des données créées aux exercices précédents.

Cet atelier constitue une bonne base pour aller plus loin, par exemple avec les déclencheurs (triggers), la gestion des erreurs dans les procédures ou les transactions.